

Software Engineering Project - *Monopoly Deal Cards Game*



Students required to elicit the requirements of the *Monopoly Deal Cards* game and implement the game using Java following the software engineering guidelines.

Evaluation Criteria:

The project will be evaluated along the following criteria:

Phase-1

- **Game requirements and Modelling (20%)**
 - User requirements and system requirements of the Game are elicited correctly.
 - The Functional and non-functional requirements identified correctly.
 - It would be useful to show that you have created use case diagrams, sequence diagrams and class diagrams of your project (for drawing the UML Diagrams: I would recommend you to use **Papyrus since it will be covered in the lab session.** However, feel free to use other tools like <https://sequencediagram.org/> and <https://app.genmymodel.com/>).

Phase-2

- **Clean design (20%)**
 - Game components are implemented appropriately, i.e. separate classes are defined to create different game components.
 - Is the project well-structured in classes and modules? Each class should perform a small set of related functionalities (No big classes, long methods)
 - Are interfaces defined and used appropriately?(Interfaces must be created for classes showing similar behaviour but not extending the same superclass)
 - Use the correct data structures
 - Adopt inheritance wisely (e.g., to distinguish between different types of players).

o Design patterns (e.g., Singleton, Facade) must be used when necessary. In this assignment, it may be necessary to use the following design patterns: Singleton, Facade, Observer, and Model-View-Controller.

- **You have to use at least five design pattern types!**

- **Functionality (40%)**

- o The functional requirements of the game are implemented correctly.
- o Consider having GUI for the game.

- **Teamwork (10%)**

- o The distribution of load between teammates is appropriate
- o Distributed version repository (Git) is used appropriately
- o Try to avoid situations where one team member did all the work and another just went for the ride. **These things can be understood very easily during the correction of your project.**
- o Make sure that your commit messages are self-explanatory and describe the type of modification that was performed in the project.
- o Commits should be performed by both team members frequently.

- **Testing (10%)**

- o Several meaningful test cases are implemented to test the main project functionalities.
- o Ensure that the main methods of each class are tested: no need to test getters and setters; ensure that essential methods (e.g., those you decided to show in the sequence diagram are tested).

Project Registration (by Week 2):

- One group project (5 people sign into groups randomly!)
- Each team member will receive the same evaluation if they have the same contribution to the project.
- **Provide the member contributions and how you divide the work. Please explicitly mention if someone have huge contribution to the project comparing to the other members. Please include a table with this information in all the submission you will made for this project.**

Important Dates:

You need to do the following for the submission:

1. Phase-1 (Deadline: Week 8: 22 April 2026)

Provide a pdf file contain the following elements:

- Game requirements.
- Provide the UML design of your project.
- Project completion plan (timeline to finish the project by Week 15).
- Provide the member contributions and how you divide the work.

2. Phase-2 (Deadline: Week 15: 9 June 2026)

Provide a pdf file contain the following elements:

- Provide any explanation you want to help me understand how you conduct the project considering the Game requirements and the concepts we discussed in the class about the best practice of being a software engineer?
- **Provide the member contributions and how you divide the work. Please explicitly mention if someone have huge contribution to the project comparing to the other members. Please include a table with this information in all the submission you will made for this project.**
- **Provide a link to your GitHub repository on Brightspace:** When submitting your assignment on Brightspace, you only need to provide a link to your Github repository in the pdf. Please, do not attempt to upload your code on Brightspace.

3. Interview (Week 15: 11-12 June 2026): You will present your work (Demo) and answer some questions form the TA and me. **More details about the timetable for the interview will be posted due to the course.**

GOOD LUCK!